Day 153 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 22, Day 7 reference

Watch: Sean Taylor - Forecasting at Scale: How and Why We Developed Prophet for Forecasting at Facebook | Forecasting with the FB Prophet Model (Rob Mulla)

`day153_prophet_style_forecaster.py`, segment by segment, in the order `main()` uses things. The changepoints, the design matrix, the penalised fit, the components, the forecast, the simulated interval, the AR(1) correction and the validation search are new. Every other part is imported unchanged from this week's exercise files; today's build has no exercise file of its own. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: t = years since the first month · c_j = changepoints · δ_j = slope changes · λ = their penalty · K = wave pairs · F = a fitted model (a dictionary)

Segment 0 — Setup

Imports: six exercise files as modules

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
sys.path.insert(0, os.path.join(HERE, "..", "week-10-lstm-gru"))
with contextlib.redirect_stdout(io.StringIO()):
    import day147_exercise as d147
    import day148_exercise as d148
    import day149_exercise as d149
    import day150_exercise as d150
    import day151_exercise as d151
    import day152_exercise as d152
AIRLINE = (0, 1, 0, 1, 12)
KS = (2, 3, 4, 5)
LAMS = (0.01, 0.1, 1.0, 10.0, 100.0, 1000.0)
HS = (1, 6, 12, 24)
```

- `import day152_exercise as d152` — Day 152's parts; it imports Week 10's LSTM, so that folder goes on the path too.
- `LAMS = (0.01, 0.1, 1.0, 10.0, 100.0, 1000.0)` — a tuple of penalties: from almost none to very stiff.
- `HS = (1, 6, 12, 24)` — the horizons printed in the tables.

Segment 1 — The parts

changepoints and design: the columns

MATH

$$\text{column } j\text{: } (t - c_j)_+ = \max(t - c_j, 0); \quad X = [1, t, \text{hinges, sin and cos waves}]$$

CODE

```
def changepoints(n, count=25, frac=0.8):
    return np.unique(np.linspace(0, int(frac * n), count + 1).round().astype(int)[1:])
def design(start, idx, cps, K):
    t = idx / 12.0
    H = np.maximum(t[:, None] - cps[None, :] / 12.0, 0.0)
    return np.column_stack([np.ones(len(idx)), t, H, d151.fourier(np.datetime64(start, "M") + idx,
        K)])
```

- `np.linspace(0, int(frac * n), count + 1)` — 26 evenly spaced numbers from 0 to 80% of n; [1:] drops the 0.
- `.round().astype(int)` — rounded to whole months.
- `np.unique(...)` — sorted, with repeats removed (a short series could round two to the same month).
- `t[:, None] - cps[None, :] / 12.0` — broadcasting: every month against every changepoint, a (months, changepoints) table.
- `np.maximum(..., 0.0)` — the hinge: negative values (before the changepoint) become 0.

fit_prophet: a penalty as extra rows

MATH

$$|y - X\beta|^2 + \lambda \Sigma \delta_j^2 = |[y; 0] - [X; \sqrt{\lambda}\, E]\, \beta|^2$$

CODE

```
def fit_prophet(y, start, K=4, lam=100.0, count=25, frac=0.8):
    n = len(y); cps = changepoints(n, count, frac) if count else np.array([], int)
    X = design(start, np.arange(n), cps, K)
    P = np.zeros((len(cps), X.shape[1])); P[:, 2:2 + len(cps)] = np.sqrt(lam) * np.eye(len(cps))
    beta = np.linalg.lstsq(np.vstack([X, P]), np.r_[y, np.zeros(len(cps))], rcond=None)[0]
    r = y - X @ beta
    return dict(beta=beta, cps=cps, K=K, lam=lam, start=start, n=n, sigma=r.std(), resid=r,
        delta=beta[2:2 + len(cps)])
```

- `changepoints(...) if count else np.array([], int)` — count 0 gives an empty integer array: no hinges, a straight line.
- `P[:, 2:2 + len(cps)] = np.sqrt(lam) * np.eye(len(cps))` — one row per changepoint, $\sqrt{\lambda}$ in that changepoint's column.
- `np.vstack([X, P])` — the penalty rows under the data rows.
- `np.r_[y, np.zeros(len(cps))]` — their targets are 0, so each row adds $\lambda\, \delta_j^2$ to the sum of squares.

components and prophet_forecast

CODE

```
def components(F, idx):
    X = design(F["start"], idx, F["cps"], F["K"]); m = 2 + len(F["cps"])
    return X[:, :m] @ F["beta"][:m], X[:, m:] @ F["beta"][m:]
def prophet_forecast(F, h):
    return sum(components(F, np.arange(F["n"], F["n"] + h)))
```

- `X[:, :m] @ F["beta"][:m], X[:, m:] @ F["beta"][m:]` — two results: the trend's columns times their coefficients, then the waves'.
- `sum(components(...))` — sum() of a pair of arrays adds them: trend + season.
- `np.arange(F["n"], F["n"] + h)` — the month positions after the last fitted month.

prophet_sims: Prophet's interval

MATH

at month k: $\Sigma_{j \le k}\, size_j\, (k - j) / 12$

CODE

```
def prophet_sims(F, h, sims=1000, seed=0):
    r = np.random.RandomState(seed)
    rate = len(F["cps"]) / F["n"]
    scale = np.mean(np.abs(F["delta"])) if len(F["delta"]) else 0.0
    size = r.laplace(0, scale, (sims, h)) * (r.rand(sims, h) < rate)
    ramp = (np.cumsum(np.cumsum(size, axis=1), axis=1) - np.cumsum(size, axis=1)) / 12.0
    return prophet_forecast(F, h) + ramp + r.randn(sims, h) * F["sigma"]
```

- `r.laplace(0, scale, (sims, h))` — a sims-by-h table of Laplace draws: sharp at 0, with long tails.
- `* (r.rand(sims, h) < rate)` — True/False becomes 1/0: most months get no change.
- `np.cumsum(np.cumsum(size, axis=1), axis=1)` — the first running sum gives the slope's total change, the second adds slopes up into levels.
- `- np.cumsum(size, axis=1)` — takes off one month, so a change at month j counts from month j + 1.

ar1_correction and choose

MATH

$\phi = \Sigma\, r_{t-1}\, r_t \,/\, \Sigma\, r_{t-1}^2;\;\; correction_h = \phi^h\, r_T$

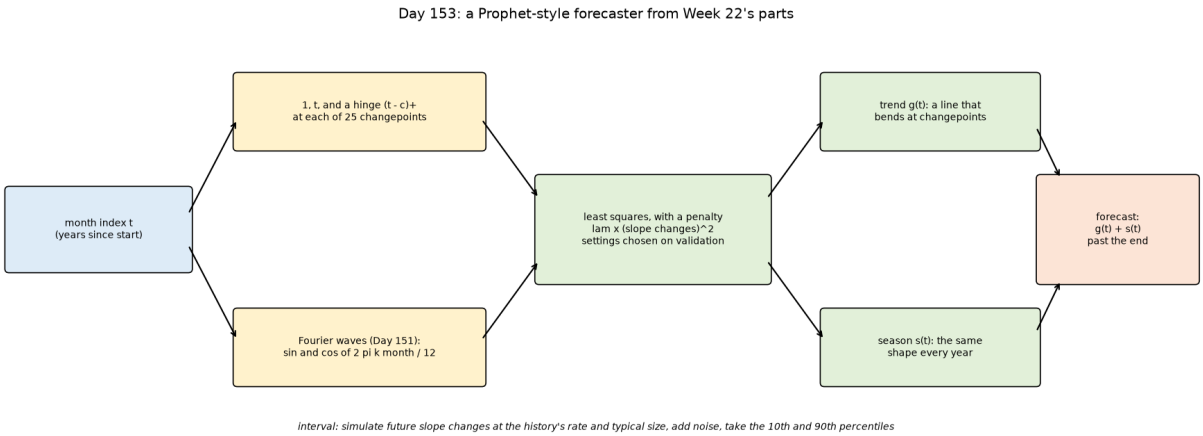
CODE

```
def ar1_correction(F, h):
    r = F["resid"]; phi = (r[:-1] @ r[1:]) / (r[:-1] @ r[:-1])
    return r[-1] * phi ** np.arange(1, h + 1), phi
def choose(y, start, origins, h=24, Ks=KS, lams=LAMS):
    E = np.array([[np.mean([np.mean(np.abs(prophet_forecast(fit_prophet(y[:o], start, K, lam), h)
        - y[o:o + h])) for o in origins])
                   for lam in lams] for K in Ks])
    i, j = np.unravel_index(np.argmin(E), E.shape)
    return Ks[i], lams[j], E
```

- `(r[:-1] @ r[1:]) / (r[:-1] @ r[:-1])` — least squares with one input and no intercept: two dot products.
- `phi ** np.arange(1, h + 1)` — phi, phi^2, ...: the correction fades month by month.

- `[[... for lam in lams] for K in Ks]` — a nested list comprehension: one row per K, one column per penalty.
- `np.unravel_index(np.argmin(E), E.shape)` — argmin counts through the flattened table; unravel_index turns that into (row, column).

SYSTEM DIAGRAM



The forecaster built from Week 22's parts.

Segment 2 — The experiments

Helpers: positions, Holt-Winters with fixed weights, the regression

CODE

```
def at(t, month):
    return int(np.flatnonzero(t == np.datetime64(month))[0])
def airline_fit(y):
    w = np.diff(y[12:] - y[:-12]); th, s2, _ = d149.fit_css(w, AIRLINE); a, b =
        d149.sarima_polys(th, AIRLINE)
    return d149.full_ar(a), a, b
def airline_forecast(fit, y, h):
    A, a, b = fit; w = np.diff(y[12:] - y[:-12])
    return d149.forecast(y, A, b, d149.css_residuals(w, a, b), h)
def fit_hw(y, season, damped):
    with np.errstate(over="ignore"):
        return d150.fit_ets(y, "add", season, damped)
def hw_forecast(F, y, h):
    _, l, b, s, _ = d150.ets_filter(y, F["alpha"], F["beta"], F["gamma"], F["phi"], *F["init"],
        F["trend_kind"], F["season_kind"])
    return d150.ets_forecast(dict(F, level=l, slope=b, season=s, n=len(y)), h)
def regression(y, start, h):
    spec = dict(lags=tuple(range(1, 14))); X, _ = d151.make_features(y, start, spec)
    return d151.recursive_forecast(y, start, spec, d151.fit_ols(X, y), h)
```

- `with np.errstate(over="ignore"):` — inside this block numpy does not warn about overflow; outside, its normal settings return.
- `dict(F, level=l, slope=b, season=s, n=len(y))` — a copy of F with these keys replaced: the fitted weights, the new states.
- `*F["init"]` — the starred tuple fills three arguments: the starting level, slope and season.

demo_check: the closed form

MATH

$(X^TX + \lambda D)\,\beta = X^Ty, \quad D = 1$ *for slope changes, 0 elsewhere*

CODE

```
F = fit_prophet(yc[:n], tc[0], K=4, lam=100.0); m = 2 + len(F["cps"]); D = np.zeros(X.shape[1]);
    D[2:m] = 100.0
b = np.linalg.solve(X.T @ X + np.diag(D), X.T @ yc[:n])
```

- `np.diag(D)` — a square matrix with D on its diagonal: the penalty only on the slope changes.
- `np.linalg.solve(A, v)` — solves A b = v directly; no inverse is formed.

demo_choose: validation origins, and a formatted header

CODE

```
first = at(tc, "1990-01"); origins = range(first - 24 - 96, first - 24 + 1); t0 =
    time.perf_counter()
K, lam, E = choose(yc, tc[0], origins)
line = np.mean([np.mean(np.abs(prophet_forecast(fit_prophet(yc[:o], tc[0], K, count=0), 24) -
    yc[o:o + 24])) for o in origins])
print(f"{len(origins)} origins, {tc[origins[0]]} to {tc[origins[-1]]} ({time.perf_counter() -
    t0:.1f} s); mean MAE (ppm) over 24 months")
print(f"{'waves':>8} " + " ".join(f"{'pen ' + format(v, 'g'):>9}" for v in LAMS))
```

- `range(first - 24 - 96, first - 24 + 1)` — 97 origins; the last one's 24 months end just before 1990-01.
- `format(v, 'g')` — the general number format: 0.01, 100, 1000, without trailing zeros.
- `f"{... :>9}"` — right-aligned in 9 characters, so the columns line up.

demo_components and demo_orders: the largest changes

CODE

```
big = np.argsort(-np.abs(F["delta"]))[:3]
rows = {"Prophet-style": f, "+ AR(1)": f + ar1_correction(F, 24)[0], "line + season":
    prophet_forecast(fit_prophet(y, te[0], K, count=0), 24),
        "airline": airline_forecast(airline_fit(y), y, 24), "Holt-Winters": d150.ets_forecast(hw,
            24),
        "regression": regression(y, te[0], 24), "LSTM avg": d152.lstm_forecast(L, y,
            24).mean(axis=0),
        "control": d152.seasonal_drift(y, te[0], 24), "last year": y[-12:][np.arange(24) % 12]}
print(f"{name}: train up to {te[cut - 1]}; validation chose {K} waves, penalty {lam:g}; 24 months,
    MAE:")
print("  " + "; ".join(f"{k} {mape(g, act):.2f}%" for k, g in rows.items()))
```

- `np.argsort(-np.abs(d))[:3]` — positions sorted by size, largest first (the minus sign reverses the order); the first three.
- `rows = {name: forecast, ...}` — a dictionary of forecasts; its order is the order written.
- `"; ".join(... for k, g in rows.items())` — every model's score in one line, separated by semicolons.

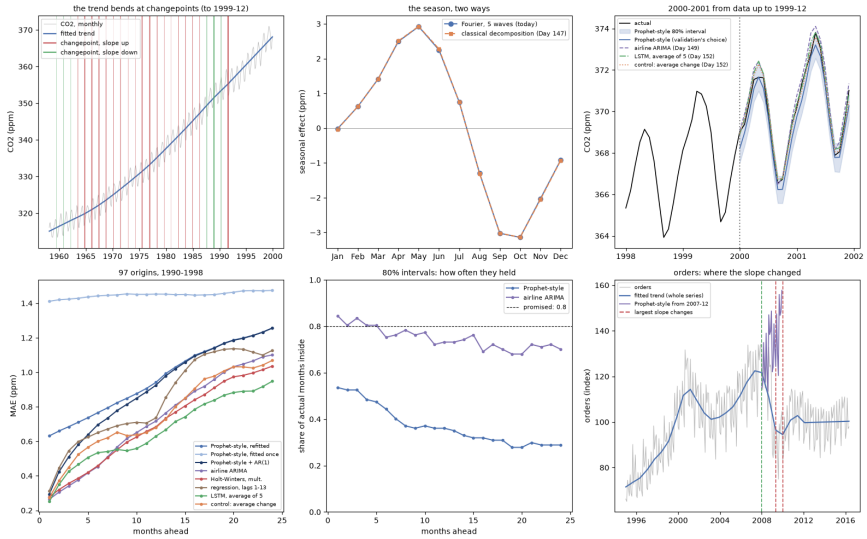
demo_rolling: a dictionary of lists, then of arrays

CODE

```
E = {k: [] for k in ("Prophet-style, refitted", "Prophet-style, fitted once", "Prophet-style +
    AR(1)", "airline ARIMA",
    "Holt-Winters, mult.", "regression, lags 1-13", "LSTM, average of 5", "control: average
        change")}
C = {"Prophet-style": [], "airline ARIMA": []}
E = {k: np.array(v) for k, v in E.items()}; C = {k: np.array(v) for k, v in C.items()}
```

- `{k: [] for k in (...)}` — a new empty list for each name; each origin appends one row of errors.
- `{k: np.array(v) for k, v in E.items()}` — each list of rows becomes an (origins, 24) array.
- `v[:, j - 1].mean()` — column j - 1 is horizon j: its mean over the origins.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.maximum(a, 0.0)` — elementwise max: the hinge.
- `a[:, None] - b[None, :]` — broadcasting into a table of every pair.
- `np.vstack([X, P])` — stack arrays as rows.
- `np.r_[y, np.zeros(k)]` — join arrays end to end.
- `np.unravel_index(i, shape)` — a flat position back to (row, column).
- `np.errstate(over="ignore")` — silence one warning inside a with block.
- `dict(F, key=value)` — a copy of a dictionary with keys replaced.
- `format(v, 'g')` — a number without needless zeros.

End of Day 153 syntax reference. For the experiments and what they showed, see the Day 153 complete guide.